

How machines learn to see — explained simply, slide by slide

This guide walks through all 39 slides of the presentation in the simplest English possible. For each slide: a small picture of it, what it actually means, the key sentence to say out loud, and answers to questions students might ask.

How to use this guide

1. Read it once, start to finish. It takes about 45 minutes.
2. Any part that makes you pause — read that section twice. The cheat-sheet slides (36–38) double as your own revision check.
3. While presenting, you do not need this guide open — the “say this” lines are the backbone, and the slides carry the rest.
4. Slide 6 is the heart of the whole talk. Practice the nine multiplications once so you can lead them confidently.

Cover — How machines learn to see

WHAT THIS ACTUALLY MEANS

This is just the title page. The talk is about computer vision: how a computer can look at a photo and tell you what is in it.

One promise to make at the start: every number in this deck is real. We really ran these experiments on a normal laptop. Nothing is made up.

How machines learn to see

SAY THIS

Today you will learn how a computer recognizes a cat, finds it in a photo, and even draws one. The only math you need is multiplying small numbers.

The plan

WHAT THIS ACTUALLY MEANS

The day has five parts, and they build on each other like a story:

Part 1: the one basic trick a computer uses to 'see' (we will do it by hand with real numbers). Part 2: how the computer learns that trick by itself, by practicing. Part 3: a shortcut — instead of teaching a computer from zero, you borrow one that someone already taught. Part 4: the cool extras — finding objects in a photo, cutting them out, talking to a model in English, and creating new images. Part 5: cheat sheets to keep.

The rule for the whole day: we never use a fancy word before explaining the idea behind it in plain words first.

THE PLAN

One day, one story

If you can multiply small numbers, you can follow every step today. New words are only used AFTER we learn them.

- Part 1 — The trick. How a pile of numbers becomes the word "cat". We compute it by hand.
- Part 2 — The learning. Nobody programs it, it practices, like the hot-and-cold game.
- Part 3 — The shortcut. Borrow a model someone already trained on a million photos.
- Part 4 — The superpowers. Find objects, cut them out, talk to images, create images.
- Part 5 — The cheat sheets. Every term and every knob, on slides you keep.

SAY THIS

One day, one story. If you can multiply small numbers, you can follow every single step.

Five starter words

WHAT THIS ACTUALLY MEANS

Before starting, we agree on five words. These are the only words used without warning later, so make sure YOU are comfortable with all five:

MODEL = the 'machine' itself. It is really just a huge collection of numbers. You put something in (a photo), it gives a guess back ('cat').

DATASET = the pile of examples we teach with. For us: lots of photos, each with its correct answer attached.

LABEL = the correct answer for one example. The photo of a cat has the label 'cat'.

TRAINING = showing the model examples again and again until its guesses become good. Like practice.

ACCURACY = a score. Out of 100 tries, how many did it get right? 90% accuracy = 90 right out of 100.

BEFORE WE START

Five words we will use all day

Only these five up front. Every other term gets introduced when you have already understood the idea behind it.

WORD	WHAT IT MEANS, IN PLAIN ENGLISH
MODEL	a machine made of numbers; input goes in, a guess comes out
DATASET	the pile of examples we teach with — photos plus their correct answers
LABEL	the correct answer attached to one example: "this photo is a cat"
TRAINING	showing the model examples, over and over, until its guesses get good
ACCURACY	out of 100 tries, how many guesses were right

SAY THIS

Read all five out loud, slowly. Tell them: every other word today gets explained before we use it.

IF THEY ASK

"Is a model a robot?" — No — it is just a very long list of numbers stored in a file, plus a recipe for using them. No body, no eyes, no brain.

(Dark slide) A computer has never seen a cat

WHAT THIS ACTUALLY MEANS

This is a dramatic pause slide. The point: a computer has no eyes. It has never 'seen' anything. So when we say it recognizes a cat, something else must be going on — and that something is just math on numbers.

This question ('so what DOES it get?') sets up the whole next slide.



SAY THIS

Pause. Ask the room: if it has no eyes, what does the computer actually get when we give it a photo? Let them guess before advancing.

A photo is just a grid of numbers

WHAT THIS ACTUALLY MEANS

This is the first big idea. If you zoom very far into any photo, it breaks into tiny squares. Each square is one **PIXEL**.

Each pixel holds **ONE** number between 0 and 255. 0 means black, 255 means white, numbers in between are shades of gray. So a black-and-white photo is literally a big grid (table) of numbers, like a spreadsheet.

A color photo is **THREE** of these grids stacked on top of each other: one grid says how red each pixel is, one how green, one how blue. Mixing red, green and blue makes every color.

The grid of numbers on the slide is **REAL** — it is the actual numbers inside the small square marked on the cat's eye.

The scary word **TENSOR** just means 'a grid of numbers, possibly several stacked'. That is all. A photo is a tensor.

PART 1: THE TENSOR

A photo is just a grid of numbers

- Zoom far enough into any photo and it falls apart into little squares.
- Each square holds one number: 0 means black, 255 means white, in-between means gray.
- A color photo is **three grids stacked** — one for red, one for green, one for blue. That is **ALL** the computer gets.

PIXEL — one tiny square of the photo
TENSOR — a grid of numbers (possibly several stacked) — nothing scarier than that



Chloe — a real photo, with a tiny square marked on her eye. Below the photo, numbers inside that square.

67	59	53	51	48	38	21
64	68	61	58	49	28	16
73	78	79	63	46	29	12
86	91	84	67	48	25	17
95	91	89	65	53	58	42

500 1000 1500 2000 2500 3000 3500 4000 4500 5000 5500 6000 6500 7000 7500 8000 8500 9000 9500 10000

SAY THIS

Ask: where are the **BIG** numbers — in the bright fur or the dark pupil? (Bright fur, because 255 = white.)
 Then the key sentence: three grids of numbers — that is **ALL** the computer ever gets.

IF THEY ASK

“Why 0 to 255?” — Computers store numbers in bytes. One byte can hold exactly 256 different values (0–255). It is just a convention.

Convolution by hand — THE most important slide

WHAT THIS ACTUALLY MEANS

Take your time here. If the students understand this slide, the rest of the day is easy.

The tiny image: a column of 2s (bright stripe) next to 0s (dark area). Where bright meets dark, there is an EDGE — a border. Question: can math FIND that edge?

The stencil: nine numbers we chose — left column is 1s (plus), right column is -1s (minus), middle is 0s.

The move: lay the stencil on the image. Multiply each image number by the stencil number sitting on top of it. Add all nine answers. That single total is the result for that position.

On the edge: the 2s sit on the +1 side, the 0s on the -1 side → total = 6, a big number. The math is shouting 'my pattern is HERE!'

On a flat dark area: everything is 0 → total = 0. The math says 'nothing here.'

So a stencil of nine numbers can DETECT an edge. This multiply-and-add move is called CONVOLUTION, and the stencil is called a FILTER (or KERNEL). Those two words sound scary but you just did the whole thing with times tables.

Part 1: The Trick

The trick, by hand — it is just times tables

The image has a bright stripe (2s) next to a dark area (0s). Where they meet is an edge. Can math FIND that edge?

1. THE IMAGE

2	2	0	0	0
2	2	0	0	0
2	2	0	0	0
2	2	0	0	0
2	2	0	0	0

2. A 3x3 STENCIL

1	0	-1
1	0	-1
1	0	-1

3. MULTIPLY EACH IMAGE CELL, ADD ALL ANSWERS

$$2 \times 1 + 2 \times 0 + 0 \times (-1) = 2$$

$$2 \times 1 + 2 \times 0 + 0 \times (-1) = 2$$

$$2 \times 1 + 2 \times 0 + 0 \times (-1) = 2$$

total = 6 — "my pattern is HERE"

Now slide the stencil onto the flat dark area; every cell is 0, every product is 0, total = 0 — "nothing here." The stencil only speaks on its own pattern.

CONVOLUTION — the multiply-and-add operation — the heart of all computer vision
FILTER (or KERNEL) — the little stencil of numbers

THE CONVOLUTION — EVERYTHING COMPUTES THE SAME PRODUCTS BUT LAYS BEFORE THE REVEAL

SAY THIS

Do the nine multiplications OUT LOUD with the room before showing the total. Then the reveal: congratulations, you just did a convolution.

IF THEY ASK

"Who chose the nine stencil numbers?" — We did, for teaching. In a real network nobody chooses them — they are learned. That is Part 2.

Slide it everywhere — feature maps

WHAT THIS ACTUALLY MEANS

One position gives one number. Now slide the same stencil across the **WHOLE** photo, stopping at every position, doing the multiply-and-add each time, and writing down each answer.

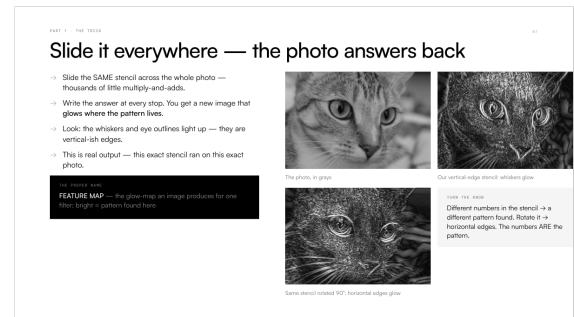
All those answers together form a new image. It is bright wherever the stencil's pattern was found, dark where it was not. This 'glow map' is called a **FEATURE MAP**.

On the slide you can see it really works: with our vertical-edge stencil, the cat's whiskers and eye outlines glow, because those are vertical-ish edges. Rotate the stencil 90 degrees and horizontal edges glow instead.

Key idea: the **NUMBERS** in the stencil decide **WHAT** pattern is found. Different numbers = different pattern.

SAY THIS

Point at the glowing whiskers: this is the real output of the exact stencil we just used, run on this exact photo.



Stride and padding

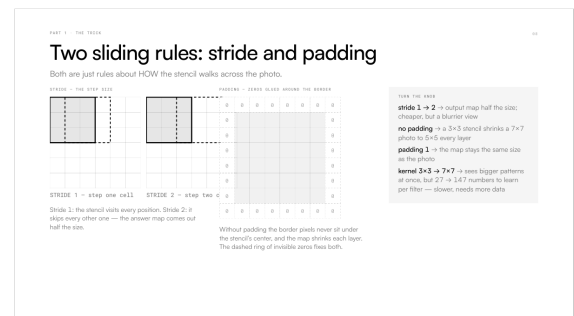
WHAT THIS ACTUALLY MEANS

Two small, boring-but-needed rules about **HOW** the stencil walks across the photo. Don't overthink these.

STRIDE = the step size. Stride 1: the stencil stops at every position. Stride 2: it jumps two cells each move, so it makes half as many stops, and the answer map comes out half the size. Faster, but a blurrier view.

PADDING = a ring of zeros glued around the photo's border.

Why? Two problems: (1) without it, the border pixels never get to sit under the **CENTER** of the stencil, so the edges of the photo get ignored; (2) the answer map comes out a bit smaller than the photo every time, so after many layers it shrinks away. The ring of zeros fixes both. The zeros are 'invisible' — they add nothing to the multiply-and-add.



SAY THIS

Both are just walking rules: stride = how big the steps are, padding = zeros around the border so the edges are not ignored. Don't linger here.

Pooling — shrink the map, forgive small shifts

WHAT THIS ACTUALLY MEANS

Take the glow map and cut it into little 2×2 windows. From each window, keep **ONLY** the biggest number and throw the other three away. This is **MAX POOLING**.

Why keep the max? The big number means 'the pattern was found here'. We only care **IF** it was found in this small area — not its exact pixel. So we keep the strongest signal per window.

Two wins: (1) the map becomes half the size, so everything after is 4× cheaper to compute; (2) if the pattern shifts by one pixel, the biggest number in the window is usually still the same — so tiny shifts of the object stop mattering. A cat slightly to the left is still a cat.

Check the example with them: top-left window holds 1, 3, 4, 8 → keep 8.

Cousins: **AVERAGE POOLING** keeps the mean instead of the max. And at the very end of a network, **GLOBAL AVERAGE POOLING** averages each whole map down to **ONE** number — making a short summary list of the photo.

POOLING — shrink the map, forgive small shifts

- Look at each little 2×2 window and **keep only the biggest number**: "was the pattern here at all?"
- The map shrinks to half size → everything after it is 4× cheaper.
- If the pattern moves by one pixel, the biggest number in the window usually does not change → **small shifts stop mattering**.
- Cousin: **average pooling** takes the mean instead. At the very end, averaging each whole map down to **ONE** number gives a short summary list of the photo.

MAX / AVERAGE POOLING → keep the max (or the mean) of each small window.

GLOBAL AVERAGE POOLING → average each whole map to one number → the final summary step.

SAY THIS

Keep each window's winner. Verify one quadrant together out loud.

ReLU — the 'keep the good news' switch

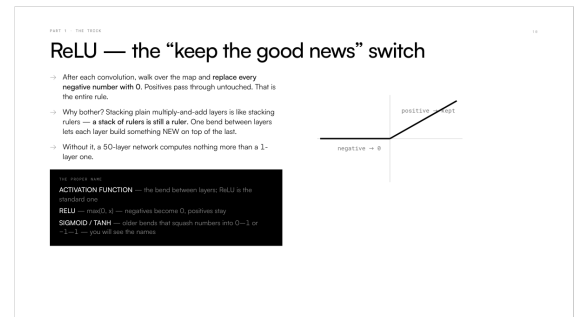
WHAT THIS ACTUALLY MEANS

The rule itself is tiny: go through the map and replace every **NEGATIVE** number with 0. Positive numbers pass through unchanged. That is literally all ReLU does.

The hard part is **WHY**. Here is the simplest version: convolution is just multiplying and adding — 'straight-line math'. If you stack many straight-line steps, the result is still one straight-line step. Like stacking rulers: a stack of rulers is still a ruler. So 50 layers would compute nothing more than 1 layer could. Depth would be fake.

Putting one little 'bend' (the ReLU) between layers breaks the straight line. Now each layer can build something genuinely **NEW** on top of the previous one. The bend is what makes deep networks actually deep.

Words: any 'bend' function between layers is called an **ACTIVATION FUNCTION**. ReLU is the standard modern one. **SIGMOID** and **TANH** are older bends — students only need to recognize the names.



SAY THIS

Anchor sentence: negatives become zero — and without that little bend, a 50-layer network is exactly as smart as a 1-layer one.

IF THEY ASK

“Why would the map have negative numbers at all?” — Stencils contain negative numbers (like our -1 column), so totals can come out negative. Negative roughly means 'the opposite of my pattern is here' — ReLU throws that away and keeps only positive evidence.

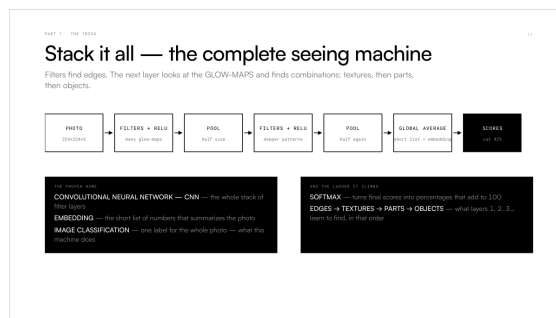
The complete CNN stack

WHAT THIS ACTUALLY MEANS

Now plug everything together into one pipeline. Photo goes in → a layer of many stencils + ReLU → pooling → another layer of stencils + ReLU → pooling → global average → final scores ('cat 92%').

The magic of stacking: layer 1's stencils look at the photo and find edges. Layer 2's stencils look at the GLOW MAPS of layer 1 — so they find combinations of edges: corners, textures (fur, stripes). Layer 3 finds combinations of textures: an ear, an eye, a wheel. The ladder goes EDGES → TEXTURES → PARTS → OBJECTS. Nobody tells the network to do this — it appears by itself.

Words on this slide: the whole stack is a CNN (convolutional neural network). The short summary list of numbers near the end is the EMBEDDING. SOFTMAX is the small last step that turns scores into percentages that add to 100. The whole job (one label per photo) is called IMAGE CLASSIFICATION.



SAY THIS

Stencils on stencils. Layer 1 finds edges, layer 2 finds textures, layer 3 finds parts. Nobody told it to — that ladder appears on its own.

Code card 1 — the CNN in PyTorch

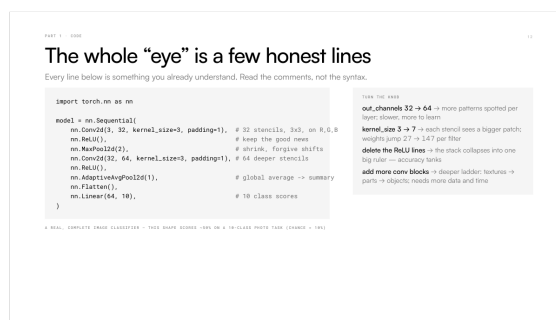
WHAT THIS ACTUALLY MEANS

Don't fear this slide — every line is a thing the students already understand. Read the COMMENTS, not the syntax.

Conv2d(3, 32, kernel_size=3) = 'make 32 stencils, each 3×3, that look at the 3 color grids'. ReLU() = the bend. MaxPool2d(2) = pooling. AdaptiveAvgPool2d(1) = global average (the summary list). Linear(64, 10) = the final scoring step that turns the summary into 10 class scores.

This is a real, complete classifier — this exact shape scored about 50% on a 10-class photo task, where blind guessing scores 10%.

The knob box is exam material: more stencils = more patterns spotted but slower; bigger stencils = sees bigger patches but many more numbers to learn; DELETE the ReLU lines and accuracy collapses (the ruler-stack problem); more layers = deeper ladder but needs more data and time.



SAY THIS

Read the comments out loud, mapping each line to the idea they already know. Then quiz them gently: what happens if I delete the ReLU lines?

Quick check 1

WHAT THIS ACTUALLY MEANS

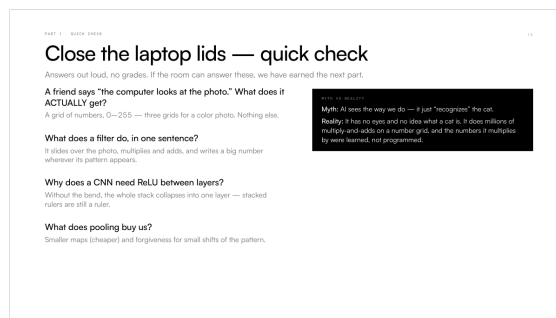
First checkpoint. Run it relaxed and out loud — popcorn style, no grades. The answers are on the slide.

The four questions test: (1) the computer only gets grids of numbers; (2) a filter slides, multiplies and adds, and lights up where its pattern is; (3) ReLU stops the stack collapsing into one layer; (4) pooling makes maps smaller and forgives small shifts.

The myth box matters: AI does NOT see like us. It has no eyes and no idea what a cat 'is'. It does millions of multiply-and-adds on a number grid.

SAY THIS

If the room struggles on the ReLU question, re-anchor with: 'the bend that makes depth real — stacked rulers are still a ruler.'

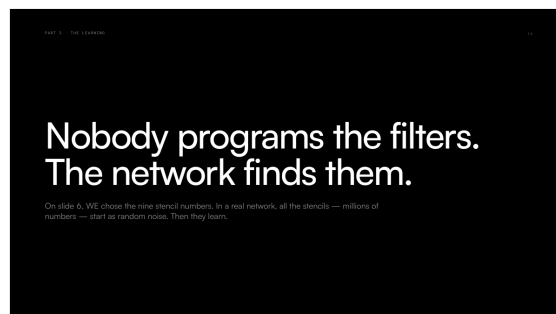


(Dark slide) Nobody programs the filters

WHAT THIS ACTUALLY MEANS

Transition into Part 2. On slide 6, WE chose the nine stencil numbers by hand. A real network has MILLIONS of stencil numbers. Nobody could choose them all by hand — and nobody does.

They start as random noise, and the network finds good values by practicing. How that practice works is the whole of Part 2.



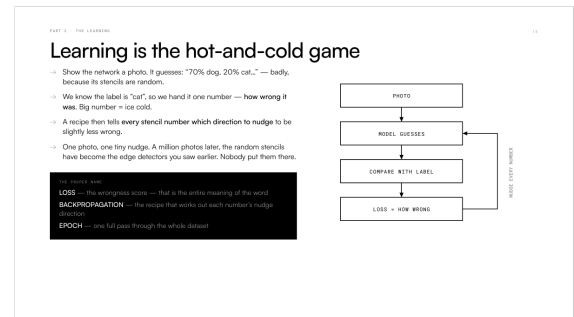
SAY THIS

One breath: 'we picked nine numbers; a real network has millions, and nobody picks them. They are learned. How?' Advance.

The learning loop (loss, backpropagation, epoch)

WHAT THIS ACTUALLY MEANS

Learning is the hot-and-cold game. The loop: (1) show the network a photo. Its stencils are random, so it guesses badly: '70% dog'. (2) We know the label is 'cat', so we calculate ONE number that says how wrong the guess was. That number is the LOSS. Big loss = ice cold. Small loss = warm. That's the entire meaning of the word 'loss': the wrongness score.



(3) A mathematical recipe then works out, for EVERY stencil number in the network, which direction to nudge it (up or down a tiny bit) to make the same mistake slightly smaller. That recipe is called **BACKPROPAGATION** — you don't need to know its inside, just its job: 'it tells every number which way to nudge'.

(4) Repeat with the next photo. One photo = one tiny nudge. After a million photos, the random stencils have turned into the edge detectors from slide 7. Nobody put them there.

An **EPOCH** = one full pass through the whole dataset (every photo seen once).

SAY THIS

Guess → compare with the answer → nudge every number a tiny bit → repeat. That is all training is.

Gradient descent and learning rate

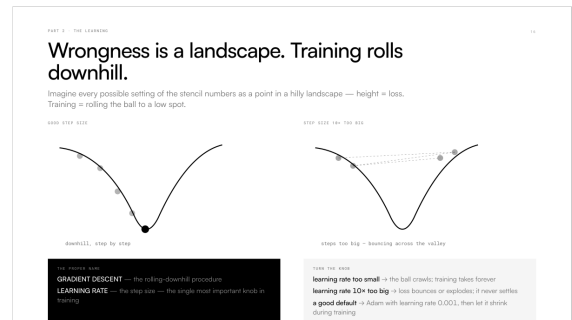
WHAT THIS ACTUALLY MEANS

A picture for the nudging. Imagine every possible setting of all the stencil numbers as one point in a landscape with hills and valleys. The HEIGHT of each point = the loss (the wrongness) you'd get with that setting. Good settings live in the valleys.

Training = placing a ball somewhere random and rolling it DOWNHILL, step by step. That procedure is GRADIENT DESCENT ('gradient' just means slope — follow the slope down).

The LEARNING RATE is the SIZE of each step. It is the single most important setting in training. Too small: the ball crawls — training takes forever. Too big (right picture): the ball leaps right over the valley, bounces back and forth across it forever, and never settles. The loss 'explodes'.

A safe default the students can quote: the Adam optimizer with learning rate 0.001, shrinking the rate as training goes.



SAY THIS

Wrongness is a landscape, training rolls a ball downhill, and the learning rate is how big each step is.

IF THEY ASK

“How does the ball know which way is downhill?” — Calculus gives the slope at the current point for every number at once. In code it is one line: `loss.backward()`. You never compute it by hand.

Overfitting — memorizing is not understanding

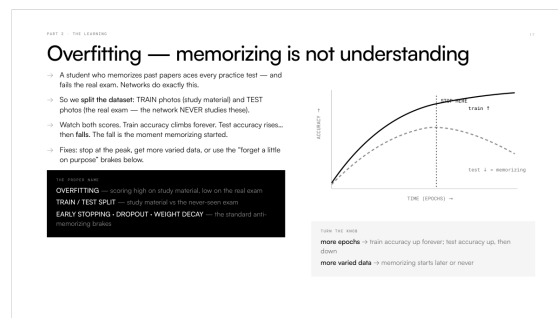
WHAT THIS ACTUALLY MEANS

Picture a student who memorizes last year's exam papers. Perfect on the practice papers — fails the real exam, because the real exam has new questions. Networks do exactly this: with enough practice on the same photos, they start memorizing those exact photos instead of learning what a cat looks like. That is **OVERFITTING**.

The protection: split the dataset in two. TRAIN photos = study material. TEST photos = the real exam — the network **NEVER** trains on these, we only use them to measure honestly.

The graph: train accuracy keeps climbing forever (memorizing always helps the study set). Test accuracy rises, peaks, then **FALLS**. The moment it falls is the moment memorizing started. So: stop at the peak.

Names to recognize (not to deeply learn): **EARLY STOPPING** (stop at the peak), **DROPOUT** and **WEIGHT DECAY** (built-in 'forget a little on purpose' brakes), and getting more varied data — the best fix of all.



SAY THIS

Never grade a model on its study material. Watch the test score, and stop when it peaks.

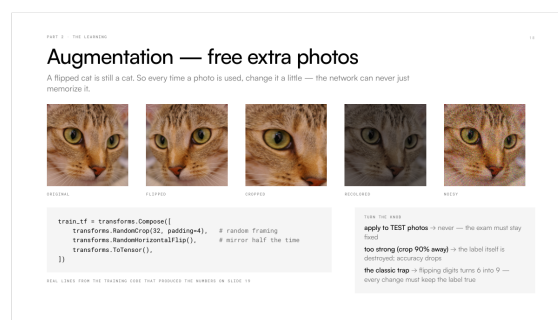
Augmentation — free extra photos

WHAT THIS ACTUALLY MEANS

A flipped photo of a cat is still a cat. A slightly cropped one too. So: every time a training photo is used, change it a little at random — flip it, shift the framing, tweak the colors. The network can never memorize an exact photo because it never sees the same photo twice. It is forced to learn the **IDEA** of the cat instead. This is **AUGMENTATION** — basically free extra training data.

The code lines on the slide are the real ones from our own training run.

Two traps to mention: (1) **NEVER** augment the test photos — the exam must stay fixed. (2) Every change must keep the label true: flipping a digit 6 turns it into a 9 — that augmentation would destroy the label.



SAY THIS

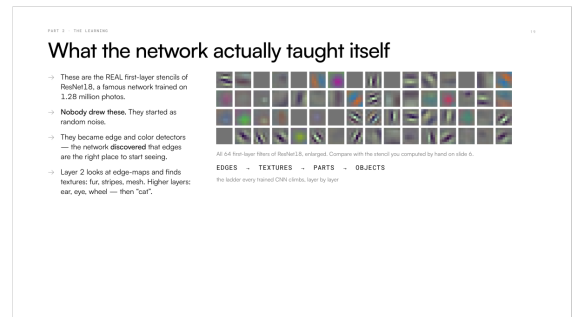
A flipped cat is still a cat — so flip, crop and recolor on the fly, and memorizing becomes impossible.

What the network taught itself

WHAT THIS ACTUALLY MEANS

This slide is proof that the whole story is true. You are looking at the REAL first-layer stencils of ResNet18, a famous network trained on 1.28 million photos.

Every one of these little patches started as pure random noise. After training, they became edge detectors and color detectors — strikingly similar to the stencil we built BY HAND on slide 6.



Nobody drew them. The network discovered, by rolling downhill on the loss, that edges are the right place to start seeing. Layer 2 (not shown) finds textures like fur and stripes; higher layers find ears, eyes, wheels.

SAY THIS

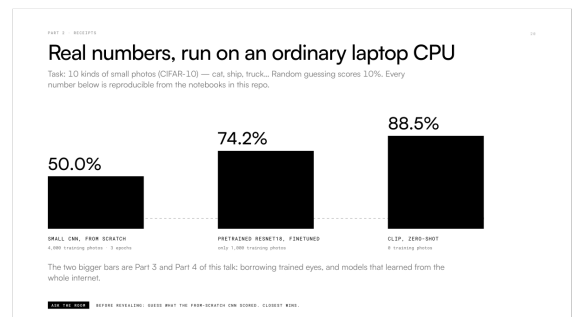
Nobody drew these. They started as random noise — and training turned them into the same kind of stencil we built by hand.

RECEIPTS — the three numbers to remember

WHAT THIS ACTUALLY MEANS

The most quotable slide of the deck. The task: classify small photos into 10 classes (cat, ship, truck...). Blind guessing scores 10%. Everything ran on a normal laptop CPU, and is reproducible from our repo.

Bar 1: a small CNN trained from scratch on 4,000 photos → 50.0%. Bar 2: a borrowed (pretrained) ResNet18, gently retrained on only 1,000 photos → 74.2%. Bar 3: CLIP, with ZERO training photos → 88.5%.



Sit with that: fewer photos, better score — and then NO photos, best score. The rest of the talk explains those two bigger bars: Part 3 = borrowing trained eyes, Part 4 = models that learned from the whole internet.

SAY THIS

Before revealing, ask the room to guess the from-scratch score — closest wins. Then: remember these three numbers — 50, 74, 88.5 — with 4,000, 1,000, and ZERO photos.

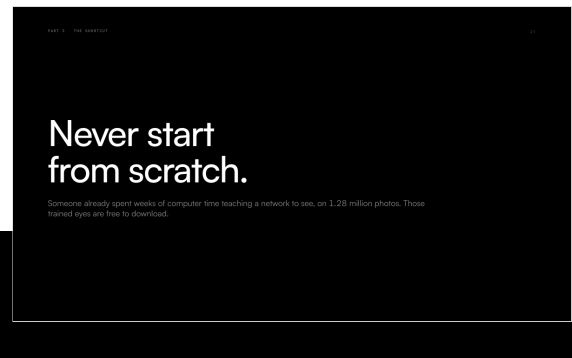
(Dark slide) Never start from scratch

WHAT THIS ACTUALLY MEANS

Transition into Part 3. Someone already spent weeks of computer time training a network to see, on 1.28 million photos. Those 'trained eyes' are free to download. So in practice, you almost never train from zero.

SAY THIS

One breath, advance.

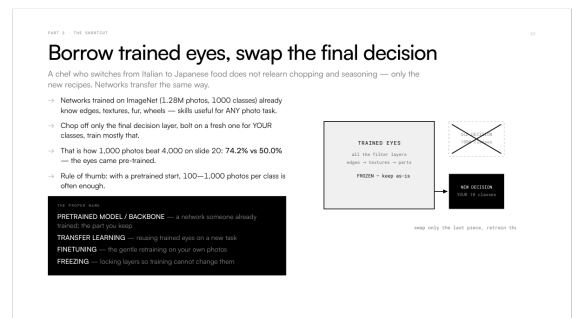


Transfer learning — borrow trained eyes

WHAT THIS ACTUALLY MEANS

The chef story: a chef who switches from Italian to Japanese cooking does not relearn how to chop and season — only the new recipes. Networks transfer the same way.

A network trained on ImageNet (1.28 million photos, 1000 classes) already knows edges, textures, fur, wheels — skills useful for ANY photo task. So: keep all the filter layers ('the trained eyes'), chop off only the final decision layer, bolt on a fresh one for YOUR classes, and train mostly just that new piece.



That is exactly how 1,000 photos beat 4,000 on the receipts slide: 74.2% vs 50.0%. The eyes came pre-trained.

Words: the downloaded network = PRETRAINED MODEL or BACKBONE. Reusing it = TRANSFER LEARNING. The gentle retraining = FINETUNING. Locking layers so training can't change them = FREEZING.

Rule of thumb worth quoting: with a pretrained start, 100–1,000 photos per class is often enough.

SAY THIS

Keep the trained eyes, swap the final decision layer. That is how 1,000 photos beat 4,000.

ResNet — why deeper suddenly worked

WHAT THIS ACTUALLY MEANS

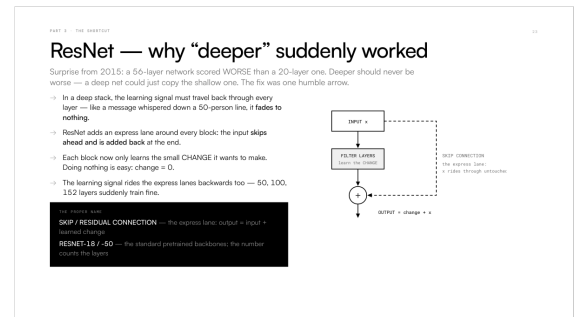
A real story from 2015: researchers found a 56-layer network scored **WORSE** than a 20-layer one. That should be impossible — a deeper net could in principle just copy the shallow one and ignore the extra layers. Something was broken.

What was broken: during training, the 'nudge' signal must travel backwards through every layer. Like a message whispered down a line of 50 people, it fades to almost nothing by the time it reaches the early layers. Deep nets couldn't train.

The fix is one humble arrow: a **SKIP CONNECTION** (or residual connection) — an express lane around every block of layers. The input skips ahead and is simply **ADDED** back to the block's output. So $\text{output} = \text{input} + \text{the block's learned CHANGE}$.

Two consequences: each block only needs to learn the small change it wants to make (and 'do nothing' becomes easy: $\text{change} = 0$); and the learning signal can ride the express lanes backwards without fading. Suddenly 50, 100, even 152 layers train fine.

RESNET-18 and RESNET-50 are the standard pretrained backbones; the number counts the layers.



SAY THIS

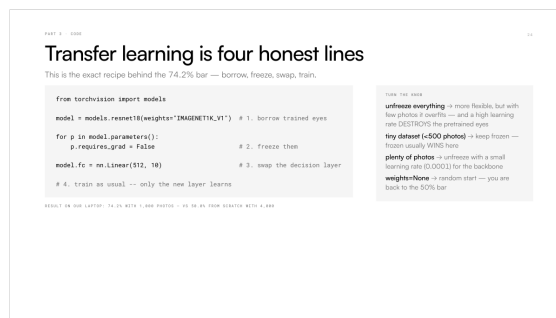
Tell the 2015 surprise first, then the fix: one express lane that adds the input back. Each block learns only the small change.

Code card 2 — transfer learning in four lines

WHAT THIS ACTUALLY MEANS

The exact recipe behind the 74.2% bar. Line 1: download ResNet18 with its ImageNet-trained weights ('borrow trained eyes'). Lines 2–3: freeze every parameter so training can't damage the eyes. Line 4: replace the final decision layer (fc) with a fresh one for our 10 classes. Then train normally — only the new layer actually learns.

Knob box (exam material): unfreeze everything = more flexible but easily overfits with few photos, and a high learning rate destroys the pretrained eyes. Under ~500 photos: keep frozen — frozen usually wins. Plenty of photos: unfreeze, but with a tiny learning rate (0.0001) for the backbone. weights=None = random start = you're back to the 50% bar.



SAY THIS

Four honest lines: borrow, freeze, swap, train.

(Dark slide) One label per photo was just the beginning

WHAT THIS ACTUALLY MEANS

Transition into Part 4. So far the machine answers one question: 'what is in this photo?' — one label per photo. The same trained eyes, with different 'heads' bolted on top, can do much more: find every object, cut them out pixel by pixel, understand English descriptions, and create brand-new images.



SAY THIS

One breath, advance.

Object detection — WHERE, not just what

WHAT THIS ACTUALLY MEANS

Detection answers 'what is WHERE'. The model outputs a LIST: for every object it finds, a rectangle around it (BOUNDING BOX), a label, and a CONFIDENCE (how sure it is). All found in one pass over the photo. The slide shows real output on our cat photo: 'cat 97%'.

Messy detail worth knowing: the raw model actually proposes MANY overlapping boxes around the same object.

A cleanup step called NMS (non-maximum suppression) keeps the most confident box of each cluster and deletes the duplicates.

How do we grade a predicted box? IoU (intersection over union): area where the predicted and true boxes OVERLAP, divided by the area they COVER TOGETHER. 1.0 = perfect match; 0.5 or more traditionally counts as a hit.

Knobs: raise the confidence cutoff → fewer false alarms but real objects get missed. Make NMS stricter → two cats sitting close together may merge into one box.

Object detection — WHERE, not just what



Real output of a detector (SSDLite) on our cat: cat 97%

→ The model outputs a list: box + label + confidence for every object it can find — all in one pass over the photo.

→ Messy detail: the raw model proposes MANY overlapping boxes per object. A cleanup step keeps each cluster's most confident box and deletes the rest.

→ How do we grade a box? Overlap with the true box: $\text{area of overlap} \div \text{area of union}$. 1.0 = perfect, ≥ 0.5 traditionally counts as a hit.

our chosen box:

BOUNDING BOX — the rectangle around an object

CONFIDENCE — how sure the model is about that box

IoU — INTERSECTION OVER UNION — the scoring scores for grading boxes

NMS — NON-MAXIMUM SUPPRESSION — the duplicate box cleanup step

TOOK THE KNOB:

confidence cutoff 0.5 → 0.8 → fewer boxes, fewer false alarms — but real objects get missed

NMS overlap limit down → stricter cleanup; two cats sitting together may merge into one box

SAY THIS

Box + label + confidence, for every object, in one pass. IoU grades a box, NMS deletes the duplicates.

Three detector families — YOLO, SSD, DETR

WHAT THIS ACTUALLY MEANS

Students only need the names, the one-line idea, and the 'character' of each. All three reuse a pretrained backbone as their eyes. All three reuse a pretrained backbone as their eyes — the difference is the head and the speed/accuracy trade.

YOLO ('You Only Look Once'): cut the photo into a grid; every grid cell predicts boxes for objects centered in it — one single pass. The speed king: 150+ photos per second on a GPU. Pick for live video, robots, drones.

SSD: same one-pass idea, but predicts from several map sizes at once — big maps catch small objects. Fast and light — this is the model that produced our cat slide. Pick for phones and small devices.

DETR: a transformer head asks about 100 learned 'questions' ('is there an object like X anywhere?'). No grid, and no NMS cleanup needed. Elegant and accurate, slower to train. Pick when accuracy beats speed.

For the exam: know the names, the one-pass idea, and that YOLO = real-time. Internals are NOT needed.

Three detector families you should recognize

All reuse a pretrained backbone as their eyes — the difference is the head and the speed/accuracy trade.

FAMILY	THE IDEA, IN PLAIN ENGLISH	CHARACTER	WHAT IT DOES
YOLO "You Only Look Once"	cut the photo into a grid; every cell predicts boxes for objects centered in it — one single pass	the speed king — 150+ photos/second on a GPU	live video, robots, drones
SSD	same one-pass idea, but predicts from several map sizes at once — big maps catch small objects	fast and light — it produced the cat slide	phones and small devices
DETR	a transformer head asks ~100 learned questions: "is there an object like X anywhere?" — no grid, no NMS cleanup needed	elegant, accurate, slower to train	accuracy over speed, modern stacks

For the exam: know the names, the one-pass idea, and that YOLO = real-time. You do not need their internals.

SAY THIS

Know the names and characters: YOLO = speed king, SSD = light for phones, DETR = the elegant transformer one.

Segmentation — a label for every pixel

WHAT THIS ACTUALLY MEANS

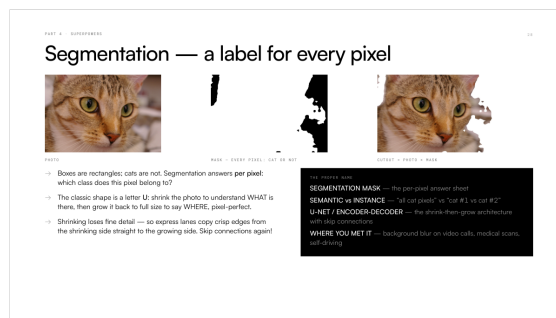
Boxes are rectangles, but cats are not. SEGMENTATION answers, for every single pixel: which class does this pixel belong to? The answer sheet — a grid where each pixel is marked 'cat' or 'not cat' — is the SEGMENTATION MASK. Multiply photo × mask and you get a perfect cutout.

Two flavors: SEMANTIC = 'all cat pixels' as one group; INSTANCE = 'cat #1's pixels vs cat #2's pixels' separately.

The classic architecture is shaped like the letter U (U-NET): the left side SHRINKS the photo (convolutions + pooling — exactly Part 1) to understand WHAT is there; the right side GROWS it back to full size to say WHERE, pixel-perfect.

Problem: shrinking loses fine detail. Fix: express lanes copy the crisp edges from the shrinking side straight across to the growing side. Skip connections again — same trick as ResNet!

Where they've met it: background blur in video calls, medical scans, self-driving cars.



SAY THIS

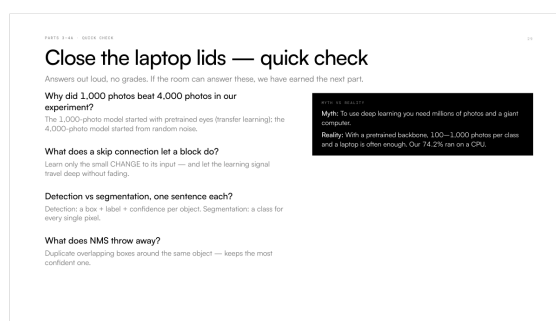
A class for EVERY pixel. Shrink to understand what, grow back to say where — and skip connections carry the sharp edges across.

Quick check 2

WHAT THIS ACTUALLY MEANS

Checkpoint for Parts 3 and 4a. Answers on the slide: (1) 1,000 beat 4,000 because of pretrained eyes; (2) a skip connection lets a block learn only the small CHANGE, and lets the learning signal travel deep without fading; (3) detection = box + label + confidence per object, segmentation = a class per pixel; (4) NMS throws away duplicate overlapping boxes.

The myth box is the most important: 'you need millions of photos and a giant computer' — FALSE. With a pretrained backbone, 100–1,000 photos per class and a laptop is often enough. Our 74.2% ran on a CPU.



SAY THIS

Run it popcorn-style. Spend the most time on the myth — it is the most empowering fact of the day.

Self-supervised learning — no labels at all

WHAT THIS ACTUALLY MEANS

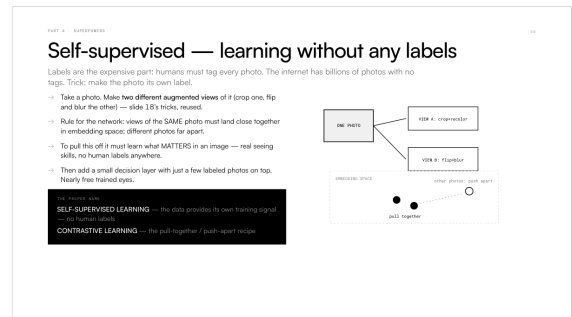
The expensive part of a dataset is not the photos — it's the LABELS, because humans must tag every photo by hand.

The internet has billions of photos with no tags. Can a network learn to see from unlabeled photos? Yes — trick: make the photo its own label.

The recipe: take one photo. Make TWO different augmented views of it (crop one; flip and blur the other — slide 18's tricks reused). Rule for the network: embeddings of views of the SAME photo must land CLOSE together; embeddings of DIFFERENT photos must land far apart. Pull together, push apart — that's why it's called CONTRASTIVE LEARNING.

To satisfy this rule, the network is forced to learn what actually MATTERS in an image (the cat-ness survives a crop and a blur; the exact pixels don't). Real seeing skills, zero human labels. This whole family is SELF-SUPERVISED LEARNING — the data supplies its own training signal.

Afterwards, add a small decision layer trained with just a few labeled photos — nearly-free trained eyes.



SAY THIS

Two augmented views of the same photo must land close together. To manage that, the network must understand images — and no human ever labeled anything.

CLIP — the model you can talk to

WHAT THIS ACTUALLY MEANS

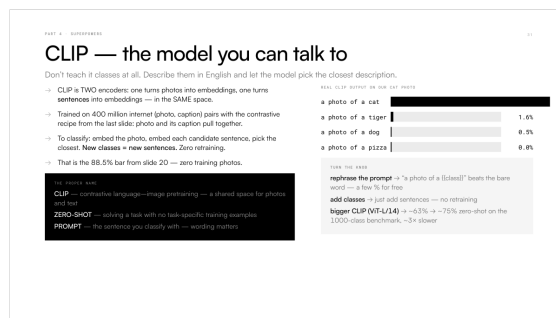
CLIP takes the contrastive trick one step further: don't teach it classes at all. Describe the classes in English.

CLIP is TWO encoders: one turns PHOTOS into embeddings, the other turns SENTENCES into embeddings — into the SAME space. Trained on 400 million (photo, caption) pairs from the internet, with the pull-together/push-apart rule: each photo and its own caption pull close.

To classify any photo: embed the photo, embed each candidate sentence ('a photo of a cat', 'a photo of a dog', ...), pick the CLOSEST sentence. Want new classes? Just write new sentences. ZERO retraining. That is ZERO-SHOT classification — solving a task with no task-specific training examples — and it is the 88.5% bar from the receipts slide.

The sentence you classify with is called the PROMPT, and its wording matters: 'a photo of a cat' scores a few % better than just 'cat', because that's what internet captions looked like.

Real output on our cat photo: cat 97.9%, tiger 1.6%, dog 0.5%, pizza 0.0%.



SAY THIS

Two encoders, one shared space for photos and sentences. To classify: pick the closest sentence. New classes = new sentences.

Code card 3 — zero-shot with CLIP

WHAT THIS ACTUALLY MEANS

Walk this one slowly — it is the complete zero-shot recipe, honest, nothing hidden. Load CLIP (photo eyes + text eyes).

Write the labels as sentences. Turn the photo into a tensor, the sentences into tokens. Encode BOTH into the same space. Measure closeness, turn into percentages with softmax (slide 11's word!).

Knobs: swap the labels list → an entirely new classifier, instantly. Phrase labels as 'a photo of a ...' → accuracy up for free. Ask something weird → 'a photo of a grumpy cat' works too; it is just a sentence.



SAY THIS

Nine lines, no training loop anywhere — and it scored 88.5%. Output on our cat: 97.9%.

GANs — a counterfeiter and a detective

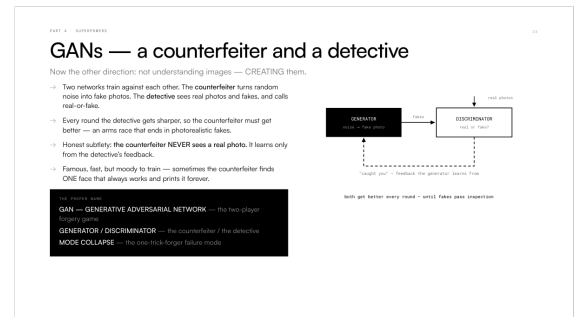
WHAT THIS ACTUALLY MEANS

Now the OTHER direction: not understanding images, but **CREATING** them. A GAN is two networks training against each other.

The **GENERATOR** (counterfeiter) turns random noise into fake photos. The **DISCRIMINATOR** (detective) is shown real photos and fakes, and must call real-or-fake. Every round the detective gets sharper, so the counterfeiter must get better — an arms race that ends in photorealistic fakes.

The honest subtlety students love: the counterfeiter **NEVER** sees a real photo. It learns only from the detective's feedback ('caught you'). It learns to paint faces without ever seeing one — only from being caught.

GANs are famous and fast, but moody to train. Classic failure: **MODE COLLAPSE** — the counterfeiter finds **ONE** face that always fools the detective and prints it forever (the one-trick forger).



SAY THIS

A counterfeiter and a detective, raising each other's game — and the counterfeiter never sees a real photo.

Diffusion — how modern image generators work

WHAT THIS ACTUALLY MEANS

DALL·E, Stable Diffusion, Midjourney — under the hood, this. And the students already know every ingredient.

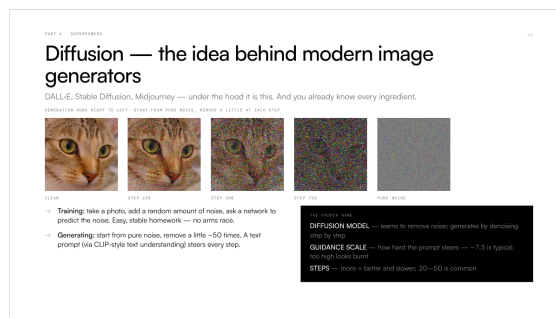
Training: take a real photo, add a random amount of noise (static), and ask a network to predict WHAT noise was added. That's it. Easy, stable homework — no arms race, no moody training.

Generating: start from PURE noise and apply the network's only skill — 'remove a little noise' — about 50 times in a row. Step by step an image emerges from the static. Read the slide's photo strip RIGHT to LEFT: pure noise → cat.

A text prompt steers every denoising step (using CLIP-style text understanding — last slide's idea reused!). That's how 'a cat astronaut on the moon' comes out.

Knobs: GUIDANCE SCALE = how hard the prompt steers (~7.5 typical; too high = burnt, oversaturated images). STEPS = more is better but slower; 20–50 is common.

GANs vs diffusion: diffusion won the image-quality war (stable training, more variety); GANs are still faster at generating.



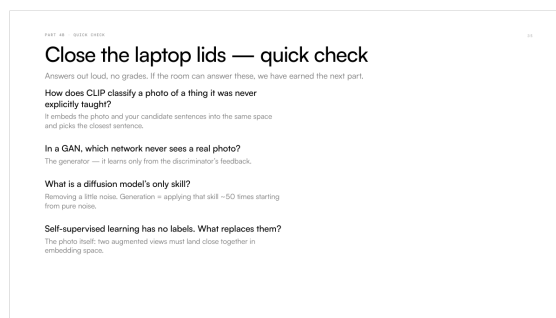
SAY THIS

Its only skill is removing a bit of noise. Generation = that skill 50 times, starting from pure static, with the prompt steering each step.

Quick check 3

WHAT THIS ACTUALLY MEANS

Final checkpoint, Part 4b. Answers on the slide: (1) CLIP classifies unseen things by embedding photo and sentences into the same space and picking the closest sentence; (2) in a GAN, the GENERATOR never sees a real photo; (3) a diffusion model's only skill is removing a little noise — applied ~50 times from pure noise; (4) in self-supervised learning, the labels are replaced by the photo itself: two augmented views must land close together.



SAY THIS

Popcorn-style again. If these land, the syllabus is covered.

Cheat sheet 1 — the syllabus map

WHAT THIS ACTUALLY MEANS

Every IOAI Section 3 topic, the slide where it was covered, and a one-line summary to remember. This is the revision index for the whole deck.

Tell the students to screenshot this slide — before the exam they can check each row and jump back to the right slide if anything feels shaky.

PART 3 - CHEAT SHEET

The IOAI syllabus map — where we covered what

Every Section 3 topic, the slide where it lives, and the one line to remember.

SYLLABUS TOPIC	SLIDE	ONE LINE TO REMEMBER
Images as tensors, pixels	85	a photo is 3 stacked grids of numbers 0–255
Convolution, filters, stride, padding	86–88	slide a stencil, multiply & add; stride = step size, padding = zero border
Pooling (max/avg/global)	89	keep each window's winner — smaller map, shift-forgiving
Activations (ReLU)	18	zero the negatives — without the bend, depth is false
CNN's & classification	11–12	stacked filters climb edges → textures → parts → objects
Loss, gradient descent, overfitting, augmentation	14–18	roll downhill on the wrongness landscape; never grade on the study set
Pretrained encoders, ResNet, transfer learning	21–23	borrow trained eyes, sweep the decision layer; skips lot depth train
Detection (YOLO/SSD/DETR), IoU, NMS	25–26	box + label + confidence in one pass; IoU grades, NMS dedupes
Segmentation, U-Net	27	a class per pixel; shrink to understand, grow back to localize
Self-supervised / contrastive	29	two views of one photo pull together — the data labels itself
CLIP & zero-shot	30–31	photos and sentences share one space; new classes = new sentences
Generative: GANs, diffusion	32–33	forgery arms race vs learn-to-denoise — diffusion won the quality war

SAY THIS

This is your revision index. Screenshot it.

Cheat sheet 2 — all 30 terms

WHAT THIS ACTUALLY MEANS

Every word 'earned' during the day, each with a one-line plain-words meaning. All 30 of them.

Useful for you too: if you can read every line of this slide and nod, you understand the deck. Any line that makes you pause — go back to that slide in this guide and re-read it.

PART 3 - CHEAT SHEET

Every word we earned today — all 30

pixel — the measure of a photo holding a number 0–255	gradient descent — nudge every number downhill on the loss
tensor — a grid (or stack of grids) of numbers	learning rate — the step size of those nudges
model — a machine made of numbers; input → guess	epoch — one full pass through the dataset
label — the correct answer attached to an example	overfitting — memorizing the study set, failing the exam
convolution — slide a stencil, multiply matching cells, add	augmentation — flip/rotate/crop — free extra photos
filter / kernel — the stencil of numbers	pretrained / backbone — a network someone already trained
feature map — the glow-map (bright = pattern found here)	transfer learning — reuse trained eyes on a new task
stride — how far the stencil steps each move	finetuning — gently retraining on your own data
padding — zeros glued around the border	skip connection — express later; output + input + change
pooling — keep each window's max (or mean) — shrink the map	bounding box / IoU / NMS — detector's rectangle, grade, and dedupe
ReLU — negatives → 0; the bend that makes depth real	segmentation mask — a class for every pixel
CNN — the full stack of filter layers	self-supervised — the data provides its own training signal
embedding — the short number list summary of an input	CLIP / zero-shot — shared photo-text space; classify by sentence
softmax — scores → percentages that sum to 100	GAN — counterfeiter vs detective
loss — the wrongness score	diffusion — generate by removing noise, step by step

SAY THIS

Thirty words, all earned, all one-liners. Screenshot.

Cheat sheet 3 — the knob table

WHAT THIS ACTUALLY MEANS

The practical-round drill table: 'change this → that happens'. The competition's practical questions usually ask exactly this — predict the effect of a change BEFORE running the code.

Practice idea for a team session: cover the right column, read a knob out loud, let everyone predict, then reveal.

PART 3 - CHEAT SHEET

The knob table — change this, get that

The practical round usually asks exactly this: predict what a change does before you run it.

THINK THIS KNOB	WHAT HAPPENS
more filters per layer (32 → 128)	richer patterns spotted; ~4× the numbers to learn and compute
bigger kernel (3×3 → 7×7)	sees wider patterns; 9 → 49 multiplications per step
stride 1 → 2	output map halves; faster, blurrier view
remove ReLU	depth collapses to one layer; accuracy tanks
learning rate ×100	loss bounces or explodes — never settles
learning rate ÷100	crawl; may never arrive in time
train far past the test-accuracy peak	overfitting — train score up, exam score down
add flip/rot/crop augmentation	typically +3–8% test accuracy on small datasets
pretrained start vs random start	our receipts: 74.2% with 1,000 photos vs 50.0% with 4,000
detector confidence cutoff up	fewer false alarms, more missed objects
better CLIP prompt wording	a few % accuracy for free — no retraining
diffusion guidance scale up (7.5 → 20)	obeys the prompt harder; colors go burnt, variety drops

SAY THIS

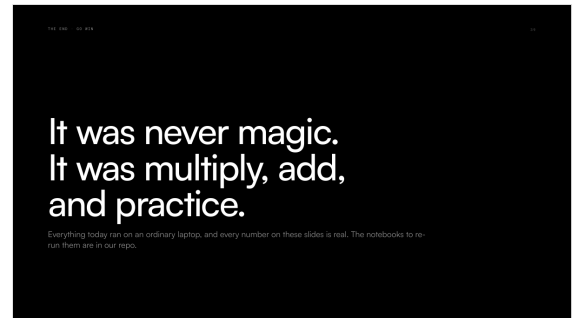
Cover the right column and quiz each other — this is exactly the practical-round skill.

Closing — it was never magic

WHAT THIS ACTUALLY MEANS

The closing message: everything in the deck was multiply, add, and practice. Everything ran on an ordinary laptop, every number was real, and the notebooks to re-run them are in the team repo.

End on the empowering note: nothing today was beyond them — they computed the core operation by hand within the first ten minutes.



SAY THIS

It was never magic. It was multiply, add, and practice. Go win.